CSC2025
6th International Conference on Soft Computing
• 19 & 20 November 2025
• Faculty of Technology and Engineering - Eset of Guilan
University of Guilan
ششمین کنفرانس بین‌المللی محاسبات نرم
• ۲۸ و ۲۹ آبان ماه ۱۴۰۴
• دانشکده فنی و مهندسی شرق گیلان

Genetic Algorithm Performance Enhancement through Adaptive Techniques and Elitism

Mohammad Hossein Taghizadeh [1], Majid Ghasemian[2]
[1] B.Sc. Student, Faculty of Technology and Engineering-Eset of Guilan, University of Guilan, Guilan, Iran;
mthz.ira@gmail.com
[2] Assistant Professor, Faculty of Technology and Engineering-Eset of Guilan, University of Guilan, Guilan, Iran;
m.ghasemian@guilan.ac.ir

ABSTRACT

This paper presents an enhanced Genetic Algorithm (GA), implemented in Python and exposed through Django API endpoints, that integrates adaptive techniques and elitism to improve efficiency and convergence speed. The proposed approach dynamically adjusts mutation and crossover probabilities based on population diversity and stagnation metrics, while an elitism mechanism ensures the preservation of top-performing solutions across generations. Using a permutation-based chromosome structure designed to evolve a target musical note sequence, the adaptive GA achieves significant improvements—reducing the required generations from 83 (with fixed rates) to 21 (with adaptive rates)—thereby minimizing local optima traps and promoting exploration. The system is further integrated with MIDO for MIDI file generation and React for visualization, enabling real-time monitoring of evolutionary progress, error magnitudes, and standard deviations. The method provides both mathematical formulations and implementation process, demonstrating how standard GAs can be transformed into robust optimization tools with applications in sequence evolution and other complex problem domains.

Keywords
Genetic Algorithm, Adaptive Mutation, Adaptive Crossover, Elitism, Population Diversity, Convergence Acceleration.

1. INTRODUCTION

Genetic Algorithms (GAs) are one of the most established metaheuristic optimization techniques, inspired by the principles of natural selection and evolutionary biology. Since their early formulation by Holland and De Jong, GAs have been successfully applied to a wide spectrum of optimization problems, ranging from combinatorial scheduling and resource allocation to machine learning, network design, and music composition [1, 2]. The flexibility of GAs makes them attractive for solving complex problems where classical optimization techniques often fail. However, conventional GAs that rely on fixed crossover and mutation rates tend to suffer from two critical drawbacks: (i) premature convergence to local optima, and (ii) slow convergence speed in high-dimensional or rugged fitness landscapes [3]. To mitigate these limitations, parameter control in evolutionary algorithms has become an active area of research. The seminal work of Srinivas and Patnaik [4] proposed adaptive probabilities of crossover and mutation based on population fitness, demonstrating significant improvements over fixed-parameter GAs. Their results highlighted the importance of balancing exploration and exploitation dynamically. Later, Eiben, Hinterding, and Michalewicz [5] provided a systematic classification of parameter control strategies into deterministic, adaptive, and self-adaptive approaches, establishing a theoretical foundation that has guided decades of subsequent research. More recent studies have further refined adaptive control mechanisms by incorporating diversity measures, stagnation metrics, or statistical properties of the evolving population [6, 7]. Basak [8] proposed a rank-based adaptive mutation scheme where mutation probabilities depend on chromosome ranking, outperforming fixed-rate and fitness-based mutation methods on multimodal optimization and traveling salesman problems. Recent study on evolutionary algorithms with adaptive genetic operators and dynamic scoring mechanisms demonstrated improved convergence and diversity in many-objective large-scale optimization tasks [9]. In the domain of program synthesis, Niani and Spector [10] showed that adaptive mutation rates significantly enhance the robustness and efficiency of genetic programming system These methods show that adaptive adjustment of GA parameters can prevent population stagnation, improve robustness, and accelerate convergence.

In parallel, the concept of elitism has been recognized as a key mechanism for enhancing GA performance. By preserving the best-performing individuals across generations, elitism ensures that valuable solutions are not lost due to random genetic operations. De Jong [2] first emphasized the role of elitism in maintaining solution quality, and later research has explored various elitist strategies such as global elitism, replace-if-better schemes, and elitist archives [11]. While elitism accelerates convergence by safeguarding high-quality individuals, it must be carefully balanced with diversity preservation, as overly strong elitism can reduce exploration capability and cause premature convergence [12].

Hybrid approaches combining adaptive parameter control and elitism have shown particular promise. Alba and Dorronsoro [13] introduced cellular and hybrid GAs that integrate local search with elitism, achieving faster convergence in large-scale optimization tasks. Other studies have demonstrated the effectiveness of integrating adaptive operators with

elitist retention in combinatorial optimization problems, revealing significant reductions in the number of generations required to reach optimal or near-optimal solutions [14]. Despite these contributions, the majority of existing work has focused on either adaptive techniques or elitism in isolation, with relatively few studies providing a unified and practical framework that exploits the synergy of both.

Another important trend in GA research is the extension to real-world, application-driven systems. For example, Katoch et al. [15] reviewed advances in GAs and emphasized their potential in domains such as sequence evolution, multimedia generation, and dynamic optimization. In the domain of creative computing, GAs have been applied to music composition and sequence generation, where adaptive parameter control helps sustain diversity and avoid repetitive structures [16]. These applications highlight the importance of practical implementations that combine algorithmic improvements with real-time visualization, reproducibility, and integration into modern software environments.

Motivated by these challenges and opportunities, this study introduces an enhanced Genetic Algorithm (GA) that combines adaptive control of crossover and mutation probabilities with a novel heuristic for dynamic elitism sizing. Unlike traditional static elitism approaches commonly discussed in GA literature, the proposed adaptive elitism strategy balances the retention of top-performing individuals according to population size and chromosome length, thereby maintaining diversity while ensuring solution quality. The algorithm is implemented in Python, exposed through Django API endpoints, and integrated with visualization tools developed in React. In this framework, mutation and crossover rates are dynamically adjusted based on population diversity and stagnation metrics, while elitism consistently preserves high-performing solutions across generations. To demonstrate its effectiveness, the method is applied to a permutation-based chromosome structure for evolving a target sequence of musical notes, achieving a significant reduction in the number of generations required compared to fixed-rate GAs. The proposed enhanced algorithm is described in Section 2, evaluated against a traditional GA in Section 3, and the conclusion are summarized in Section 4.

2. Enhanced GA Algorithm Methodology

To present the improvements introduced in this study, it is first necessary to outline the standard Genetic Algorithm (GA) framework, which serves as the baseline model. A GA is a population-based metaheuristic inspired by natural evolution, where candidate solutions (chromosomes) evolve over successive generations through selection, crossover, and mutation. Although this conventional model has been successfully applied in numerous optimization problems, it often encounters limitations such as premature convergence, reduced diversity, and inefficiency in navigating complex search spaces.

In the following subsections, the baseline GA model is described in detail, followed by the modifications proposed in this work. The enhancements include adaptive control of crossover and mutation probabilities, as well as a novel dynamic elitism sizing strategy. These refinements aim to improve exploration–exploitation balance, preserve high-performing solutions, and accelerate convergence.

A. Standard GA

The standard Genetic Algorithm (GA) is a stochastic search and optimization technique inspired by the principles of Darwinian evolution. It operates on a population of candidate solutions, represented as chromosomes, which evolve over multiple generations to approach optimal solutions. The main components of the classical GA framework are shown in Figure 1 describe as follows:

Fitness Function:

Each chromosome is evaluated using a fitness function that measures its quality relative to the optimization objective. Fitness values guide the selection process by assigning higher probabilities to better-performing individuals.

Selection:

Parent chromosomes are selected from the population based on their fitness scores. Common methods include roulette-wheel selection, tournament selection, and rank-based selection. The goal is to give higher chances of reproduction to fitter individuals while maintaining diversity.

Crossover (Recombination):

Selected parents undergo crossover, where segments of their genetic material are exchanged to produce offspring. This operator enables the exploration of new solution regions by combining characteristics of different parents.

Mutation:

Mutation introduces random changes to chromosomes, typically with a low probability, to maintain genetic diversity and prevent premature convergence to local optima.

Termination Condition:

The algorithm iterates through generations until a stopping criterion is met, such as reaching a maximum number of generations, achieving a target fitness value, or observing convergence of the population.

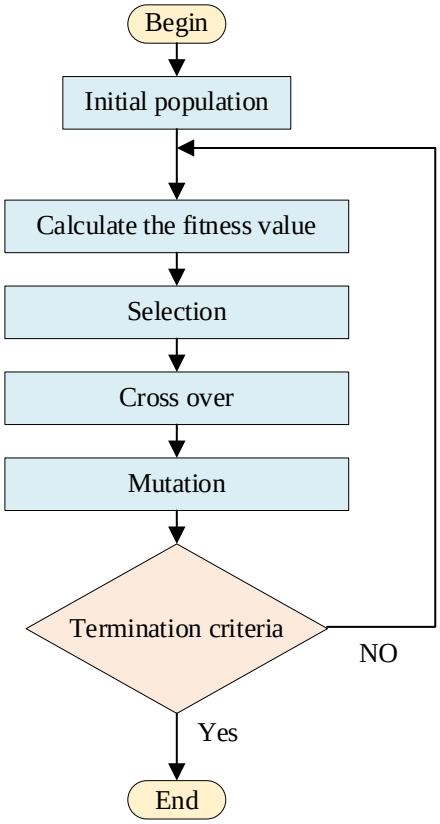

Figure 1. Standard GA algorithm [17]

B. ENHANCEMENTS TO THE GA MODEL

To overcome the limitations of the standard GA, this study introduces adaptive control of crossover and mutation probabilities together with a dynamic elitism sizing strategy. These enhancements are designed to maintain population diversity, prevent premature convergence, and ensure that the best-performing individuals are preserved across generations.

Adaptive Crossover Probability (Pc):
A fixed crossover rate in standard GA cannot adapt to different search phases, often resulting in premature convergence or insufficient exploration. In contrast, the proposed adaptive crossover strategy dynamically balances exploration and exploitation by adjusting the rate according to population diversity and fitness dynamics. At its core, crossover is scaled proportionally to diversity, enabling broad recombination when variety is high and promoting exploitation when the population converges. This adjustment is refined with several safeguards: (i) when the population approaches homogeneity (low diversity), crossover is sharply reduced to allow fine-tuning; (ii) when fitness stagnates despite moderate diversity, crossover is penalized to avoid ineffective recombination; and (iii) when genuine fitness improvements occur, crossover is reset to a high value, leveraging promising building blocks. Additional mechanisms include periodic resets to maximum crossover every fixed number of generations to counteract long-term stagnation, and improvement-rate monitoring, which halves crossover if progress falls below a defined threshold, shifting exploration pressure toward mutation. Finally, the crossover rate is clamped within valid limits, ensuring a dynamic yet stable operator that maintains balance between exploration, exploitation, and long-term adaptability. Equation (1) formalizes the proposed adaptive crossover rate.

$$P_c = P_{min} + (P_{max} - P_{min})\frac{\sigma}{\sigma_{max}} \tag{1}$$

$$\sigma_{max} = \max(F) - \min(F)$$

Where, P_c is the probability of crossover, σ representing the standard deviation (STD), F is the fitness function, σ_{max} is the maximum deviation of the fitness values, P_{min} and P_{max} represent the minimum and maximum probabilities, equal to 0 and 1, respectively.

Adaptive Mutation Probability (Pm):

Standard GA with a fixed mutation rate suffers from either excessive randomness that disrupts good solutions or insufficient diversity that causes premature convergence. In contrast, the proposed adaptive mutation strategy dynamically adjusts the rate according to population diversity and recent fitness trends. Specifically, mutation decreases as diversity increases, while in cases of diversity collapse and fitness stagnation, the rate is amplified to stimulate exploration. Conversely, when genuine progress is achieved, mutation is deliberately reduced to protect improving solutions. The final mutation value is clamped within valid bounds, ensuring stability while maintaining adaptability across generations. Equation (2) presents the proposed adaptive mutation rate that addresses these limitations.

$$P_m = P_{max} - (P_{max} - P_{min})\frac{\sigma}{\sigma_{max}} \tag{2}$$

$$\sigma_{max} = \max(F) - \min(F)$$

Where, P_c is the probability of mutation.

Dynamic Elitism Sizing:

In traditional GA, all offspring are generated through selection, crossover, and mutation, fully replacing the parent population. As a result, even the best solution discovered so far may be lost in the next generation due to stochastic operators. To address this drawback, the enhanced GA introduces an additional step: elitism. Elitism ensures that top-performing individuals are preserved across generations, preventing the loss of valuable solutions, improving population stability, accelerating convergence, and safeguarding against regression in solution quality. In the proposed method, the elitism rate is adaptively adjusted at each epoch according to Equation 3. This formulation ensures that at least one elite individual is always preserved across generations, while allowing the number of elites to grow adaptively with problem scale. The square-root scaling provides a balanced growth rate: larger mating pools and smaller chromosome lengths increase elitism size, reflecting higher confidence in selecting and preserving high-quality solutions. Conversely, when chromosomes are longer (i.e., search space more complex), elitism is moderated to maintain diversity. The scaling factor acts as a tuning parameter, enabling fine control over the strength of elitism. Overall, this dynamic rule avoids the pitfalls of static elitism, providing a balance between protecting top performers and sustaining exploration.

$$Elitism\_Size = \max\left(1, \left\lfloor SF.\sqrt{\frac{MPZ}{CL}} \right\rfloor\right) \tag{۳}$$

Where,

- SF is the scaling factor,
- MPZ is the population mating pool size,
- CL is the chromosome length.

Implementation Overview:

The enhanced GA integrates the above adaptive mechanisms into the evolutionary loop. The algorithm is implemented in Python, exposed via Django API endpoints, and integrated with React-based visualization tools. This allows real-time monitoring of population dynamics, including diversity, mutation/crossover probabilities, and elite retention across generations.

3. PERFORMANCE EVALUATION: STANDARD GA VS. ENHANCED GA

To demonstrate the performance of the proposed enhanced GA, the standard GA is first implemented on the sample problem using the parameters shown in Figure 2. This figure also illustrates the React-based application interface, which was developed to facilitate easier implementation and user interaction. In this baseline configuration, adaptive mutation and crossover mechanisms are disabled, with both rates fixed at constant values. Furthermore, the algorithm is executed without the elitism option.

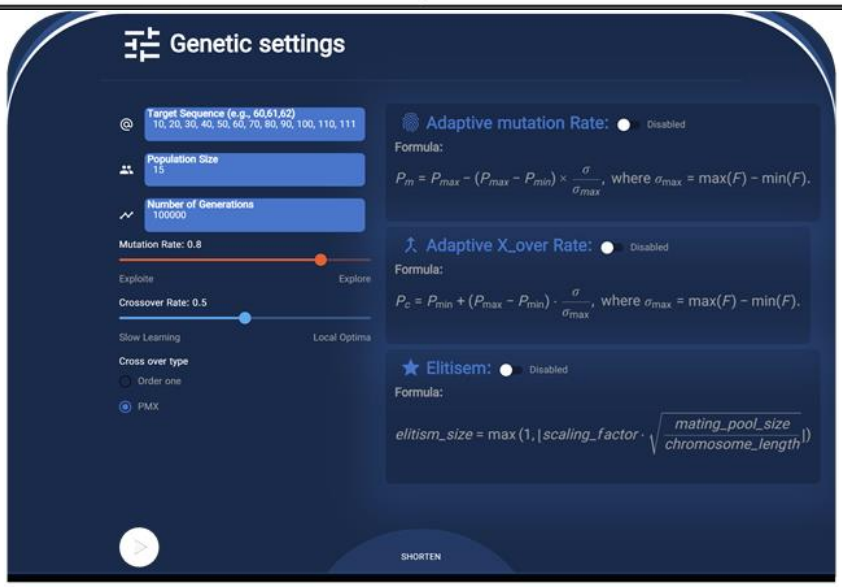

Figure 2. React interface for tuning the standard GA

Different generations of the implemented standard GA are presented in Table 1. As noted earlier, no adaptivity is applied in this case; thus, mutation and crossover rates remain fixed across all generations, and the absence of elitism results in a constant mating pool size. Based on the error progression over successive generations, it can be observed that in some cases the algorithm becomes trapped in local minima, causing the error to remain unchanged. In certain generations, the error even increases, further demonstrating the limitations of the standard GA.

Table 1 .Generation procedure of the standard GA

Generation	Mutation Rate	Crossover Rate	Mating pool size	Elite Error
1	0.800	0.500	15	9
2	0.800	0.500	15	7
3	0.800	0.500	15	6
4	0.800	0.500	15	6
5	0.800	0.500	15	6
6	0.800	0.500	15	6
7	0.800	0.500	15	5
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
28	0.800	0.500	15	2
29	0.800	0.500	15	2
30	0.800	0.500	15	3
31	0.800	0.500	15	3
32	0.800	0.500	15	3
33	0.800	0.500	15	4
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
75	0.800	0.500	15	2
76	0.800	0.500	15	2
77	0.800	0.500	15	2
78	0.800	0.500	15	2
79	0.800	0.500	15	2
80	0.800	0.500	15	2
81	0.800	0.500	15	2
82	0.800	0.500	15	2
83	0.800	0.500	15	0

Figure 3 shows the standard deviation (STD) of the population, while Figure 4 illustrates the maximum deviation of the population and the error across different generations. Although STD can significantly influence the convergence speed of the algorithm, the lack of adaptivity and elitism leads to the need for many more iterations to achieve convergence. Moreover, for a considerable portion of the process, the error remains unchanged even when the STD varies.

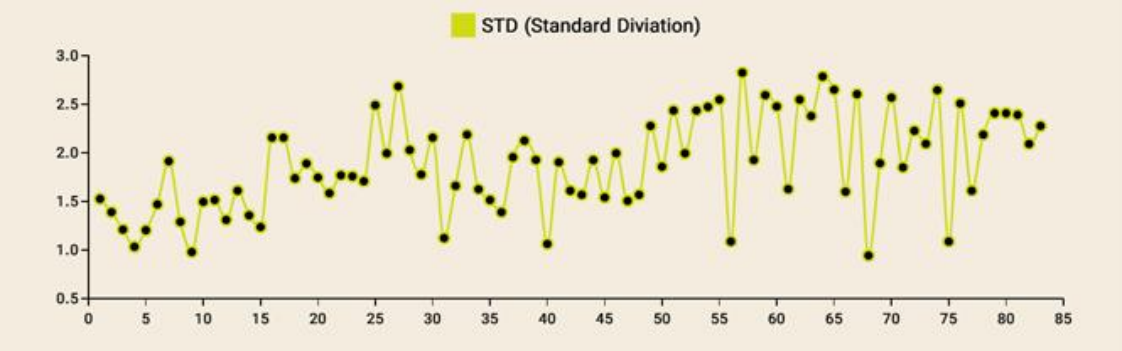

Figure 3. STD of mating pools at each iteration for standard GA

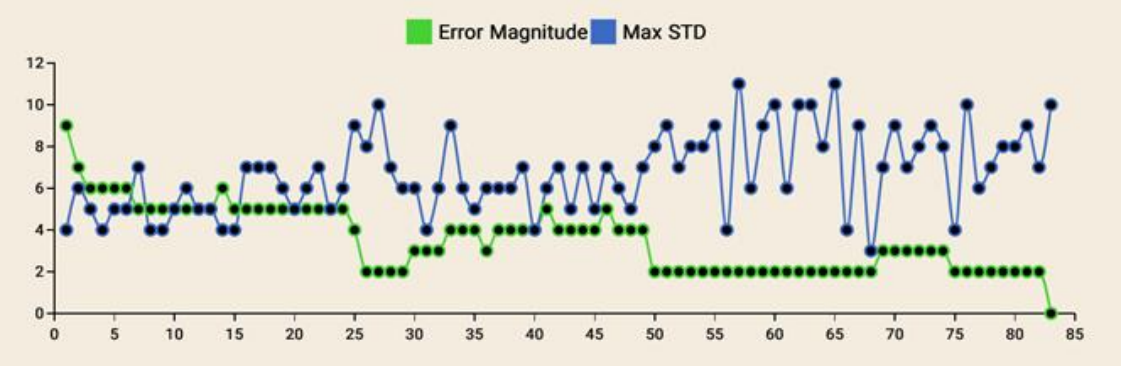

Figure 4. Max. STD of mating pools and error at each iteration for standard GA

The provided interface for the enhanced GA implementation is shown in Figure 5, where the adaptive mutation and crossover rates are enabled, and elitism is activated.

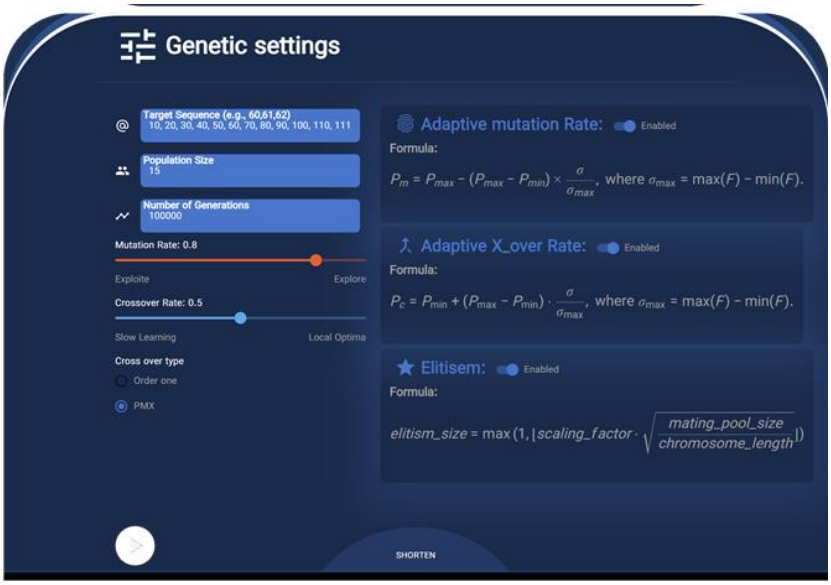

Figure 5. GA armed with additivity and elitism

Table 2 presents the results of implementing adaptivity and elitism in the GA procedure. It can be observed that by incorporating intelligent adaptivity, the process becomes more efficient than the standard GA and reaches the endpoint within 21 iterations. The results demonstrate that when the error decreases, the model sustains progress by increasing the crossover rate and decreasing the mutation rate. Conversely, when the process stagnates at a certain error value, it alters the search path by increasing the mutation rate and reducing the crossover rate. Furthermore, the use of elitism increases the mating pool size in each iteration, which contributes to faster convergence of the process.

Table 2. Generation procedure of the enhanced GA

GENERATION	MUTATION RATE	CROSSOVER RATE	MATING POOL SIZE	ELITE ERROR
1	0.800	0.500	17	8
2	0.660	0.276	19	8
3	0.682	0.127	21	8
4	0.648	0.144	23	7
5	0	0.950	25	6
6	0	0.950	27	5
7	0	0.950	29	4
8	0	0.950	31	4
9	0.664	0.136	33	4
10	0.738	0.098	35	4
11	0.850	0.475	37	4
12	0.796	0.136	39	4
13	0.859	0.109	41	4
14	0.804	0.133	43	3
15	0	0.950	45	3
16	0.740	0.475	47	3
17	0.695	0.120	49	2
18	0	0.950	51	2
19	0.707	0.114	53	2
20	0.655	0.140	55	2
21	0.808	0.475	57	0

Figure 6 and Figure 7 illustrate the variation of STD and maximum STD across iterations and their impact on problem convergence. For example, after iteration 8, the error becomes fixed at a certain value, where a low mutation rate and a high crossover rate lead to reduced STD within the population. However, in the proposed method, STD is subsequently increased by raising the mutation rate and reducing the crossover rate, which results in a decrease in error at iteration 13 and allows the algorithm to reach the final solution in less time.

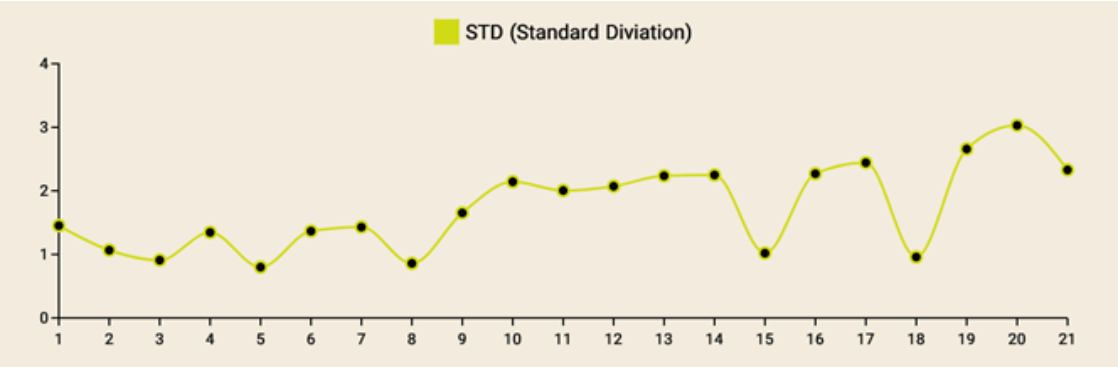

Figure 6. STD of mating pools at each iteration for enhanced GA

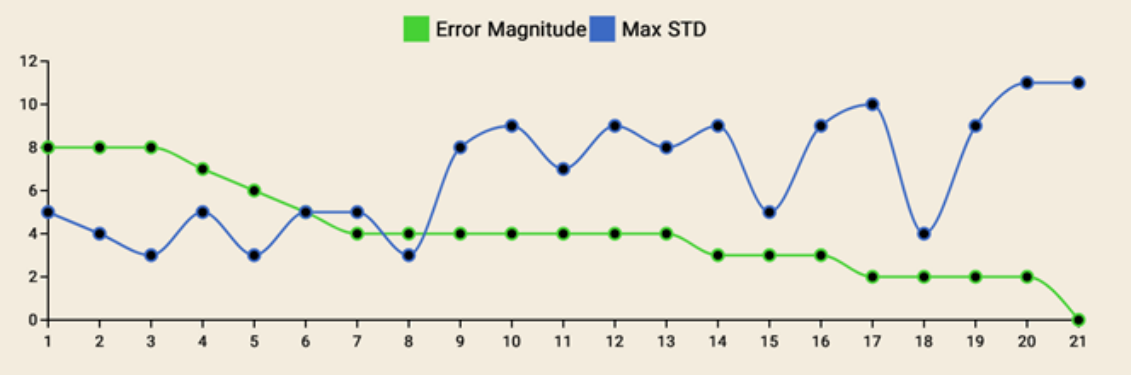

Figure 7. Max. STD of mating pools and error at each iteration for enhanced GA

5. CONCLUSIONS

This study proposed an enhanced Genetic Algorithm (GA) that integrates adaptive control of crossover and mutation rates with a dynamic elitism sizing strategy. Unlike traditional GAs that rely on fixed parameters, the proposed approach leverages population diversity and fitness trends to regulate operators in real time, while preserving high-quality solutions through adaptive elitism. The method was implemented in Python, exposed via Django API endpoints, and integrated with React-based visualization tools for interactive monitoring. Experimental results on a sequence evolution problem demonstrated that the enhanced GA significantly outperformed the standard GA, reducing the required generations from 83 to 21, accelerating convergence, and avoiding premature stagnation in local minima. Despite its demonstrated advantages, this work opens several avenues for future research. First, the proposed adaptive mechanisms can be extended and validated on a broader set of benchmark problems, including high-dimensional and multimodal optimization tasks. Second, hybridization with other metaheuristics (e.g., Particle Swarm Optimization, Differential Evolution, or Simulated Annealing) may further improve robustness and search efficiency. Third, exploring self-adaptive parameter tuning—where the parameters themselves evolve as part of the chromosome—could enhance flexibility beyond the current rule-based adaptivity. Finally, integration of advanced visualization and explain ability tools could provide deeper insight into evolutionary dynamics, supporting both research and practical deployment in complex real-world domains such as scheduling, control systems, and creative computing. Overall, the proposed adaptive GA framework demonstrates that combining adaptive operators with elitism can transform standard GAs into more powerful, stable, and versatile optimization tools.

6. REFERENCES

[1] J. H. Holland, "Adaptation in natural and artificial systems," University of Michigan Press google schola, vol. 2, pp. 29-41, 1975.
[2] A. de Jong Kenneth, "Analysis of the behavior of a class of genetic adaptive systems," Technical Report No. 185, Department of Computer and Communication Sciences, University of Michigan, 1975.
[3] "Goldberg, DE: Genetic Algorithms in Search, Optimization & Machine Learning, 401pp., Addison-Wesley (1989)," vol. 7, no. 1, pp. 168-168, 1992.
[4] M. Srinivas and L. M. Patnaik, "Adaptive probabilities of crossover and mutation in genetic algorithms," IEEE Transactions on Systems, Man, and Cybernetics, vol. 24, no. 4, pp. 656-667, 2002.
[5] Á. E. Eiben, R. Hinterding, and Z. Michalewicz, "Parameter control in evolutionary algorithms," IEEE Transactions on evolutionary computation, vol. 3, no. 2, pp. 124-141, 2002.
[6] X. Yao, "Evolving artificial neural networks," Proceedings of the IEEE, vol. 87, no. 9, pp. 1423-1447, 1999.
[7] J. Zhang, H. S.-H. Chung, and W.-L. Lo, "Clustering-based adaptive crossover and mutation probabilities for genetic algorithms," IEEE Transactions on evolutionary Computation, vol. 11, no. 3, pp. 326-335, 2007.
[8] A. Basak, "A rank based adaptive mutation in genetic algorithm," arXiv preprint arXiv:2104.08842, 2021.
[9] X. Wang, W. Zhao, J.-N. Tang, Z.-B. Dai, and Y.-N. Feng, "Evolution algorithm with adaptive genetic operator and dynamic scoring mechanism for large-scale sparse many-objective optimization," Scientific Reports, vol. 15, no. 1, p. 9267, 2025.
[10] A. Ni and L. Spector, "Effective adaptive mutation rates for program synthesis," 2024, pp. 952-960.
[11] H. Ishibuchi, N. Tsukamoto, and Y. Nojima, "Evolutionary many-objective optimization: A short review," in 2008 IEEE congress on evolutionary computation (IEEE world congress on computational intelligence), 2008: IEEE, pp. 2419-2426.
[12] C. M. Fonseca and P. J. Fleming, "Genetic algorithms for multiobjective optimization: formulationdiscussion and generalization," 1993, vol. 93, July ed., pp. 416-423.
[13] E. Alba and B. Dorronsoro, "Introduction to cellular genetic algorithms," in Cellular Genetic Algorithms: Springer, 2008, pp. 3-20.
[14] C. A. C. Coello, "Constraint-handling techniques used with evolutionary algorithms," 2022, pp. 1310-1333.

[15] S. Katoch, S. S. Chauhan, and V. Kumar, "A review on genetic algorithm: past, present, and future," Multimedia tools and applications, vol. 80, no. 5, pp. 8091-8126, 2021.

[16] G. Papadopoulos and G. Wiggins, "AI methods for algorithmic composition: A survey, a critical view and future prospects," 1999, vol. 124: Edinburgh, UK, pp. 110-117.

[17] M. A. Albadr, S. Tiun, M. Ayob, and F. Al-Dhief, "Genetic Algorithm Based on Natural Selection Theory for Optimization Problems," Symmetry, vol. 12, no. 11, doi: 10.3390/sym12111758.

[18] Y. Wang, K. Habib, A. Wadood, and S. Khan, "The Hybridization of PSO for the Optimal Coordination of Directional Overcurrent Protection Relays of IEEE Bus System. Energies 2023, 16, 3726," ed, 2023.

[19] Y.-C. Lin and H.-D. Yeh, "Identifying Groundwater Contaminant Source Characteristics Using Simulated Annealing," Journal of the Chinese Institute of Environmental Engineering, vol. 16, no. 3, pp. 183-188, 2006.